Linux and bash (1/2)

System, processes, services, logs, users: the things you keep googling.

Processes

`top`	live: M mem, P cpu, q quit
`htop`	friendlier top (install it)
`ps aux | grep nginx`	find a process
`pgrep -a nginx`	PID + command
`kill PID`	polite stop (SIGTERM)
`kill -9 PID`	force kill (SIGKILL)
`pkill -f name`	kill by name
`cmd &`	run in background
`nohup cmd &`	survive logout
`jobs / fg / bg`	shell jobs, Ctrl+Z pauses

Services: systemctl

`systemctl status X`	state + last log lines
`systemctl start X`	start now
`systemctl stop X`	stop now
`systemctl restart X`	stop + start
`systemctl reload X`	reread config, no downtime
`systemctl enable --now X`	start now + on boot
`systemctl disable X`	not on boot
`systemctl is-active X`	script-friendly check
`systemctl --failed`	what is broken
`systemctl daemon-reload`	after editing a unit file
`systemctl edit X`	override unit safely

Logs: journalctl

`journalctl -u X`	logs of one service
`journalctl -u X -f`	follow live
`journalctl -u X -n 100`	last 100 lines
`journalctl --since "1 hour ago"`	time filter
`journalctl -p err -b`	errors since boot
`journalctl -xe`	jump to end, with hints
`journalctl --disk-usage`	how big are logs

Resources

`uptime`	load avg: 1 / 5 / 15 min
`free -h`	RAM and swap
`df -h`	disk space per mount
`du -sh * | sort -h`	what is eating space
`lsblk`	disks and partitions
`lscpu / nproc`	CPU info / core count
`uname -a`	kernel + arch
`cat /etc/os-release`	which distro
`vmstat 1`	cpu/mem/io each second
`dmesg -T | tail`	kernel msgs: OOM, disk

Load avg above core count (nproc) = overloaded.

Users and permissions

`whoami / id`	who am I, my groups
`sudo -u bob cmd`	run as another user
`chmod 644 f`	rw-r--r--
`chmod +x run.sh`	make executable
`chown bob:dev f`	set owner:group
`chown -R bob dir`	recursive
`useradd -m -s /bin/bash bob`	new user + home
`usermod -aG docker bob`	add to group
`passwd bob`	set password
`last`	recent logins

r=4 w=2 x=1. 755 = rwxr-xr-x (owner, group, others).
Re-login after changing groups.

Network peek (more on Networking page)

`ss -tulpn`	what listens on which port
`lsof -i :80`	who owns port 80
`ip a`	my IP addresses

Notes

Linux and bash (2/2)

Find, filter, transfer, connect, automate.

Find and search

`find . -name "*.log"`	by name
`find . -type f -mtime -1`	changed in last 24h
`find . -size +100M`	big files
`find . -name "*.tmp" -delete`	find + delete
`grep -rn "text" .`	search inside files
`grep -i / -v / -c`	ignore case / invert / count
`grep -E "a|b"`	regex or
`grep -A2 -B2 "x"`	lines after / before
`which cmd / type cmd`	where a command lives
`locate name`	fast index search

Text tools

`tail -50 f`	last 50 lines
`tail -f app.log`	follow live
`less f`	scroll, /search, q quit
`wc -l f`	count lines
`sort | uniq -c | sort -rn`	count + rank duplicates
`cut -d, -f1,3 f`	CSV columns 1 and 3
`awk '{print $1}' f`	print column 1
`sed 's/old/new/g' f`	replace text
`sed -i 's/a/b/' f`	edit file in place
`cmd | xargs rm`	turn lines into args
`diff -u a b`	compare files
`cmd | tee out.txt`	show and save

Archives and transfer

`tar -czf a.tgz dir/`	create archive
`tar -xzf a.tgz`	extract
`tar -tf a.tgz`	list contents
`unzip a.zip`	unzip
`scp f user@host:/tmp/`	copy over SSH
`rsync -avz src/ host:dst/`	smart, resumable sync
`curl -O url`	download file
`wget url`	download file

SSH

`ssh user@host`	connect
`ssh -i key.pem user@host`	with key file
`ssh -p 2222 user@host`	custom port
`ssh-keygen -t ed25519`	make a key pair
`ssh-copy-id user@host`	install your key
`ssh -L 8080:localhost:80 host`	tunnel a port
`chmod 600 key.pem`	ssh refuses loose keys

~/.ssh/config: Host alias / HostName / User / IdentityFile

Bash tricks

`a && b`	b only if a succeeded
`a || b`	b only if a failed
`cmd > f  |  cmd >> f`	overwrite | append
`cmd 2>&1 | less`	include errors
`cmd > /dev/null 2>&1`	silence everything
`$(cmd)`	use output inline
`!!  |  sudo !!`	repeat last | as root
`Ctrl+R`	search history
`Ctrl+A / Ctrl+E`	line start / end
`Ctrl+L / Ctrl+D`	clear / log out
`echo $?`	last exit code, 0 = ok
`export VAR=value`	set env variable

Script skeleton

```
#!/usr/bin/env bash
set -euo pipefail   # stop on error
name="${1:-world}"  # arg + default
if [ -f "$f" ]; then echo ok; fi
for f in *.log; do gzip "$f"; done
while read -r l; do echo "$l"; done < in
```

Cron and packages

`crontab -e`	edit your jobs
`*/5 * * * * cmd`	every 5 minutes
`0 2 * * * cmd`	daily at 02:00
`min hr day mon weekday`	field order
`sudo apt update && sudo apt upgrade`	Debian/Ubuntu refresh
`sudo apt install X`	install
`sudo dnf install X`	RHEL/Fedora

Notes

Git

Daily flow, branches, undo, conflicts.

git pull = fetch + merge (or rebase). git restore file brings back the committed version of a file.

Setup

`git config --global user.name "N"`	set your name
`git config --global user.email "e"`	set your email
`git init`	new repo here
`git clone URL`	copy a repo
`git remote -v`	show remotes
`git remote add origin URL`	link a remote

Daily

`git status -sb`	short status
`git add -p`	stage hunk by hunk
`git add .`	stage everything
`git commit -m "msg"`	save a snapshot
`git commit --amend`	fix last commit
`git diff | git diff --staged`	unstaged | staged
`git log --oneline --graph -15`	compact history
`git show SHA`	one commit
`git blame file`	who changed each line

Branches

`git switch -c feat`	create + switch
`git switch main`	switch
`git branch -a`	list all
`git branch -d feat`	delete merged
`git merge feat`	merge into current
`git rebase main`	replay on top of main
`git cherry-pick SHA`	copy one commit
`git push -u origin feat`	first push + track
`git push origin --delete feat`	delete remote branch

Sync

`git fetch --prune`	get updates, change nothing
`git pull --rebase`	update, keep history linear
`git push`	upload commits
`git push --force-with-lease`	safer force push

Undo: the "oh no" table

`git restore f`	discard edits to f
`git restore --staged f`	unstage f
`git reset --soft HEAD~1`	undo commit, keep changes
`git reset --hard HEAD~1`	undo + DELETE changes
`git revert SHA`	safe undo on shared branch
`git reflog`	find "lost" commits
`git reset --hard SHA`	jump back to a reflog entry
`git clean -fd`	delete untracked files

Stash and tags

`git stash -u`	shelve work, incl. new files
`git stash list`	see the shelf
`git stash pop`	bring it back
`git tag v1.0 && git push --tags`	release marker
`git log -S"text"`	who added/removed text
`git bisect start`	binary-search a bug

Merge conflict, step by step

```
1  open file, find <<<<<<< ======= >>>>>>>
2  keep the right lines, delete markers
3  git add file
4  git merge --continue  (or rebase)
   escape hatch: git merge --abort
```

Never force-push or rebase shared branches.
Commit small, message says why.

Notes

Docker and Compose

Build, run, inspect, clean up, and multi-container stacks.

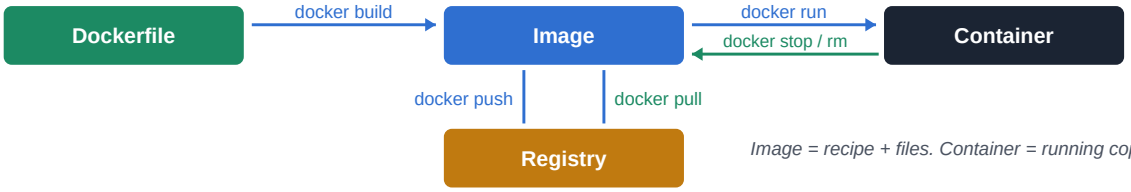


Image = recipe + files. Container = running copy.

Images

`docker build -t app:1.0 .`	build from ./Dockerfile
`docker images`	list local images
`docker pull nginx:alpine`	download
`docker tag app:1.0 user/app:1.0`	rename for registry
`docker push user/app:1.0`	upload
`docker rmi IMAGE`	delete image

Run

`docker run -d --name web nginx`	background + name
`-p 8080:80`	host:container port
`-it ubuntu bash`	interactive shell
`--rm`	delete when it exits
`-e KEY=val`	env variable
`-v $(pwd):/app`	mount folder
`-v data:/var/lib/x`	named volume
`--network mynet`	join a network
`--restart unless-stopped`	auto-restart

Manage and debug

`docker ps  |  docker ps -a`	running | all
`docker stop|start|restart web`	lifecycle
`docker rm -f web`	force delete
`docker logs -f --tail 100 web`	follow logs
`docker exec -it web sh`	shell inside
`docker inspect web`	full JSON details
`docker stats`	live CPU / RAM
`docker cp web:/f .`	copy file out

Cleanup

`docker system df`	what uses space
`docker system prune`	remove unused stuff
`docker image prune -a`	remove unused images
`docker volume ls`	list volumes
`docker volume prune`	delete unused volumes
`docker network ls`	list networks

Dockerfile skeleton

```
FROM python:3.12-slim
WORKDIR /app
COPY requirements.txt .   # deps first
RUN pip install -r requirements.txt
COPY . .                  # cache friendly
EXPOSE 5000
CMD ["python", "app.py"]
```

Compose

`docker compose up -d`	start stack
`docker compose up -d --build`	rebuild + start
`docker compose down`	stop + remove
`docker compose down -v`	also delete volumes
`docker compose ps`	status
`docker compose logs -f web`	follow one service
`docker compose exec web sh`	shell in service
`docker compose config`	validate file

compose.yaml skeleton

```
services:
  web:
    build: .
    ports: ["8080:5000"]
    environment: [DB_HOST=db]
    depends_on: [db]
  db:
    image: postgres:16
    volumes: [data:/var/lib/postgresql/data]
volumes:
  data:
```

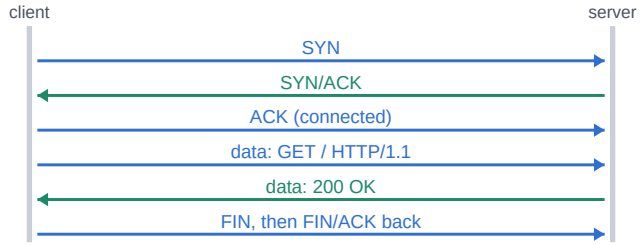
Fixes: port already allocated: change host port. Exits instantly: docker logs. Cannot reach db: use service name, not localhost. Permission denied on socket: sudo usermod -aG docker $USER, then re-login.

Notes

Networking

Ports, subnets, DNS, HTTP, and a debugging ladder.

TCP handshake and close



Debug commands

`ping -c 4 host`	reachable? latency
`mtr host`	live traceroute
`traceroute host`	hops to host
`dig example.com`	DNS lookup
`dig +short A example.com`	just the IPs
`dig @8.8.8.8 example.com`	ask a specific DNS
`dig -x 1.2.3.4`	reverse lookup
`nc -zv host 443`	is the port open?
`curl -I url`	headers only
`curl -v url`	full request/response
`curl -s -o /dev/null -w "%{http_code}" url`	just the status
`ss -tulpn`	what listens here
`ip a  |  ip r`	addresses | routes
`tcpdump -i eth0 -nn port 80`	capture packets

"Cannot reach it" ladder

```
1 ping IP        network path ok?
2 dig name        DNS resolves?
3 nc -zv IP port  port reachable?
4 curl -v url     app answers?
5 ss -tulpn       (on server) listening?
6 firewall: ufw status / cloud security group
```

Common ports

`22`	SSH	`389`	LDAP
`53`	DNS	`445`	SMB
`80`	HTTP	`3389`	RDP
`443`	HTTPS	`3306`	MySQL
`25`	SMTP	`5432`	Postgres
`143`	IMAP	`6379`	Redis
`21`	FTP	`27017`	MongoDB
`123`	NTP	`8080`	alt HTTP

Subnets (CIDR)

`/24`	`255.255.255.0`	254 hosts
`/25`	`.128`	126 hosts
`/26`	`.192`	62 hosts
`/27`	`.224`	30 hosts
`/28`	`.240`	14 hosts
`/30`	`.252`	2 hosts
`/16`	`255.255.0.0`	65,534 hosts

hosts = 2^(32 - n) - 2. Private: 10/8, 172.16/12, 192.168/16.
127.0.0.1 loopback. 169.254.x.x = DHCP failed.

DNS records

`A / AAAA`	name to IPv4 / IPv6
`CNAME`	alias to another name
`MX`	mail server
`TXT`	SPF, verification
`NS`	who serves the zone
`PTR`	IP to name

HTTP status

`200 / 201`	OK / created
`301 / 302`	moved permanently / temp
`400 / 401`	bad request / not logged in
`403 / 404`	forbidden / not found
`429`	rate limited
`500 / 502`	app crashed / bad upstream
`503 / 504`	overloaded / upstream timeout

Notes